MouseHash Transformation Path Design

A canonical transformation taxonomy for role-bundled neural data analysis

Prepared for Maria Kesa / MouseHash

1. Core idea

MouseHash should not treat tools as functions that directly consume raw roles. A neuroscience dataset may contain neural data, stimuli, behavior, conditions, time organization, and metadata, but scientific tools usually require an analysis-ready view derived from those roles.

The central design pattern is: Roles -> Transformations -> AnalysisView -> Tool -> Artifact. Roles describe what exists. Transformations describe how it becomes analyzable. Tools describe the scientific operation. Artifacts remember the result and provenance.

The rule is deliberately strict: tools should not secretly reshape, align, bin, filter, normalize, split, or aggregate data. These operations should be explicit, typed, and provenance-tracked transformations.

Design slogan: Roles tell MouseHash what exists. Transformations tell MouseHash how it becomes analyzable. Tools tell MouseHash what scientific question to ask.

2. Three-level analysis object model

Layer	Meaning	Examples
Roles	Scientific material present in the dataset.	Neural data, stimuli, behavior, conditions, time organization, metadata.
Views	Analysis-ready objects materialized from roles by transformations.	Trial x neuron matrix, stimulus x trial x neuron tensor, design matrix, RDM, latent trajectory.
Tools	Scientific operations applied to views.	PCA, GLM, decoder, RSA, HMM, noise correlations, report generator.

3. Canonical MouseHash transformation families

These transformation families are meant to be first-class objects in MouseHash, with their own specs, assumptions, failure modes, and provenance. The list is intentionally broad enough to cover electrophysiology, calcium imaging, behavior, video, stimulus models, and multimodal DANDI/NWB-style datasets.

#	Family	Purpose	Examples
1	Selection / subsetting	Choose the slice of data to analyze.	select_sessions; select_brain_regions; select_neurons; select_trials; exclude_bad_trials; select_time_window
2	Synchronization / clock correction	Make clocks and timestamps comparable across devices.	sync_device_clocks; correct_timestamp_drift; map_frame_index_to_time; resolve_trial_event_order
3	Alignment	Map streams onto a shared scientific reference event.	align_to_stimulus_onset; align_to_choice; align_behavior_to_neural_time; align_sessions
4	Segmentation / epoching	Cut continuous recordings into scientifically meaningful chunks.	make_trials; make_epochs; make_sliding_windows; make_baseline_windows; make_event_locked_segments

5	Binning / resampling	Change temporal resolution or sample grid.	bin_spikes; downsample_lfp; resample_behavior; aggregate_calcium_frames; interpolate_missing_samples
6	Filtering / smoothing	Remove or emphasize temporal/spatial components.	lowpass_filter; highpass_filter; bandpass_filter; notch_filter; smooth_spike_counts; gaussian_smooth_psth
7	Normalization / scaling	Make values comparable while avoiding leakage.	zscore_neurons; baseline_subtract; center_features; whiten_features; normalize_embeddings; train-only scaling
8	Quality control / artifact rejection	Detect and remove unreliable units, trials, channels, sessions, or samples.	detect_bad_channels; detect_bad_units; detect_motion_artifacts; detect_dropped_frames; detect_timing_mismatches
9	Feature extraction	Turn raw stimuli, behavior, or neural streams into usable features.	extract_vit_embeddings; extract_clip_embeddings; extract_pose_features; extract_lfp_bandpower; extract_spike_count_features
10	Labeling / annotation	Create categorical, semantic, anatomical, or task variables.	assign_condition_labels; assign_animate_inanimate_labels; assign_choice_labels; assign_brain_region_labels; assign_cell_type_labels
11	Aggregation / summarization	Reduce observations into response summaries or descriptive statistics.	average_trials; average_by_stimulus; compute_psth; compute_tuning_curve; compute_response_window_mean
12	Residualization / nuisance removal	Remove known explanatory factors before another analysis.	subtract_stimulus_mean; regress_out_running_speed; regress_out_session_effects; remove_global_signal; remove_batch_effects
13	Splitting / evaluation resampling	Create train/test/validation logic and null resamples.	train_test_split; k_fold_split; leave_one_session_out; time_blocked_split; bootstrap_resample; permutation_shuffle
14	Tensorization / view construction	Materialize the object consumed by a tool.	make_trial_by_time_by_neuron_tensor; make_design_matrix; make_lagged_design_matrix; make_population_state_matrix
15	Dimensional transformation	Change representation space before another tool.	pca_transform; nmf_transform; cebra_transform; autoencoder_encode; project_onto_semantic_axis
16	Distance / similarity construction	Turn objects into pairwise structure.	compute_rdm; compute_kernel_matrix; compute_graph_adjacency; compute_correlation_distance; compute_cosine_similarity
17	Event detection	Create discrete events from continuous signals.	detect_spikes; detect_calcium_events; detect_saccades; detect_licks; detect_movement_onsets; detect_ripples
18	Coordinate / unit conversion	Convert measurements into shared units or reference systems.	samples_to_seconds; frames_to_seconds; pixels_to_degrees_visual_angle; probe_coordinates_to_brain_regions
19	Missing-data handling	Represent, remove, or impute	drop_missing; mask_missing;

		incomplete observations.	interpolate_missing_behavior; impute_neural_features; align_only_complete_trials
20	Artifact packaging	Save durable outputs and register them for provenance.	save_model_artifact; save_tensor_artifact; save_metric_table; save_html_report; register_artifact_in_datajoint

4. Transformation-aware tool contract

Each tool should declare not only its required roles, but also the exact view it consumes and the transformations that are allowed to produce that view. This prevents hidden preprocessing choices from re-entering the system as haunted notebook spells.

Contract field	Meaning
requires_roles	Which canonical roles must exist: neural_data, stimuli, behavior, conditions, time_organization, metadata.
requires_view	The exact analysis-ready object consumed by the tool, e.g. observation x neuron matrix or stimulus x trial x neuron tensor.
allowed_transformations	Which transformation recipes can produce the view safely and meaningfully.
default_recipe	A conservative exploratory default, not a hidden scientific decision.
assumptions	What must be true for the result to be meaningful.
failure_modes	How the transformation or tool can mislead, break, leak information, or invent structure.
validation_checks	Controls and uncertainty procedures that should accompany the analysis.
provenance_required	Parameters, versions, hashes, source paths, split identity, and artifact paths required for reproducibility.

5. Example contract: stimulus-aligned ridge encoding

```
analysis_move: stimulus_aligned_ridge_encoding
roles:
  required:
    - neural_data
    - stimuli
    - time_organization
  optional:
    - behavior
    - metadata

transformations:
  - quality_control.select_good_units
  - synchronization.map_stimulus_frames_to_neural_clock
  - alignment.align_to_stimulus_onset
  - segmentation.extract_response_window
  - aggregation.mean_response_in_window
  - feature_extraction.extract_vit_embeddings
  - tensorization.make_design_matrix
  - splitting.k_fold_by_stimulus
  - normalization.zscore_train_only

tool:
  name: fit_ridge_encoding_model
  consumes_view:
    X: observations x stimulus_features
    Y: observations x neurons

artifact:
  - model_weights
  - cross_validated_scores
```

```
  - prediction_plots
  - structure_discovery_report
```

6. Example analysis moves as transformation paths

Analysis move	Role target	Transformation path	Artifacts
Stimulus-aligned ridge encoding	Neural data + stimuli + time organization	QC good units -> synchronize clocks -> align to stimulus onset -> segment response window -> aggregate response -> extract ViT embeddings -> make design matrix -> k-fold split by stimulus -> train-only normalization -> fit ridge encoding	model weights; CV scores; prediction plots; report
Trial-averaged PCA by stimulus	Neural data + stimuli/conditions + time organization	select trials -> align to stimulus onset -> extract response window -> group by stimulus identity -> average trials -> z-score neurons -> make condition x neuron matrix -> run PCA	scores; loadings; explained variance; visualization
Noise correlations	Neural data + conditions/stimuli + time organization	align trials -> make stimulus x trial x neuron tensor -> subtract stimulus mean response -> compute residuals -> compute pairwise correlations	noise-correlation matrix; QC table; figure
Temporal decoder	Neural data + behavior/conditions + time organization	align neural and behavior clocks -> make lagged time windows -> build train/test split -> standardize train-only -> fit decoder	decoder model; held-out performance; confusion/trajectory plot
Representational similarity analysis	Neural data + stimuli/conditions + optional behavior/metadata	make response matrix -> aggregate by stimulus/condition -> normalize -> compute neural RDM -> compute stimulus/model RDM -> compare RDMs with null control	RDMs; correlation statistics; permutation p-values; report

7. DataJoint-backed implementation spine

The DataJoint schema should record transformations as scientific computation objects, not as incidental preprocessing code. Raw/imported role tables remain stable; computed transformation artifacts can be regenerated; tool artifacts are tied to the exact view and transformation lineage that produced them.

Object	Role in MouseHash	Type
RoleBundle	Canonical six-role mapping: conditions, stimuli, behavior, neural data, time organization, metadata.	Imported/manual
TransformationSpec	Parameters and assumptions for a transformation: window, bin size, filters, scaling, split scheme, labels.	Manual / lookup
TransformationRun	Execution record for the transformation, including code version, input hashes, output paths, and status.	Computed
AnalysisView	Analysis-ready object produced by transformations: matrix, tensor, design matrix, RDM, feature table, or trajectory.	Computed artifact
ToolSpec	The tool contract: required roles, required view, assumptions, failure modes, validation plan, output artifact.	Manual / lookup
ToolRun	Bounded scientific operation applied to an AnalysisView.	Computed
AnalysisArtifact	Model, metric table, plot, latent factors, connectivity graph, statistical result, or	Computed artifact

	report bundle.	

8. Reproducibility rules

- No tool may hide alignment, binning, filtering, normalization, splitting, aggregation, or tensorization inside an opaque implementation.
- Every transformation has a spec object: parameters, source roles, output view type, assumptions, and failure modes.
- Every evaluative workflow must distinguish train-fitted transformations from test-applied transformations to avoid leakage.
- Every analysis artifact stores the transformation lineage, code version, parameter spec, source data identity, split identity, and artifact paths.
- Exploratory views and split-safe evaluative views should be different artifacts, even if they use similar code.
- The same algorithm applied to different views is a different scientific move.

9. Practical MouseHash rule

The unit of analysis should be an AnalysisMove, not only a tool:

```
AnalysisMove = RoleSignature
             + TransformationPlan
             + AnalysisView
             + Tool
             + ValidationPlan
             + Artifact
```

This turns MouseHash from a bag of neuroscience algorithms into a typed, queryable, reproducible workflow system. It also gives agents a stable target: first inspect roles, then choose or build the required view, then run the bounded tool, then package the result into an auditable artifact.